A Mini Project Report of

PCC-201-COM : Data Structures

Submitted by

Purva Pramod Raut (30)

Under the Guidance of

Prof. Tejal H. Patil



Department of Computer Engineering

BHARATI VIDYAPEETH'S
COLLEGE OF ENGINEERING, LAVALE, PUNE, 412115

Savitribai Phule Pune University

[2026-27]

BHARATI VIDYAPEETH
COLLEGE OF ENGINEERING, LAVALE, PUNE

CERTIFICATE

This is to certify that the "**Mini Project Report**" submitted by **Purva Pramod Raut (30),** **"Implementation of Snake and Ladder [BFS]"** is work done by her and submitted during 2026–2027 academic year, in partial fulfillment of the requirements for the award of the degree of **Bachelor of engineering** in computer Engineering, at **Bharati Vidyapeeth's College of Engineering, Lavale, Pune**.

Prof. Tejal H. Patil
(Guide)

Dr. Lalit V. Patil
Head of Computer Engg. Department

ACKNOWLEDGEMENT

It gives immense pleasure to express our gratitude & thanks to all those who have helped in the timely completion of the Mini-Project work. We have a great pleasure in expressing our deep sense of gratitude and indebtedness to **Prof. Tejal H. Patil** Guide for their invaluable guidance, constant supervision and sustained encouragement throughout the tenure in this Mini Project work.

We would like to express our thanks to **Dr. Lalit V. Patil**, Head of Department, Department of Computer Engineering Bharati Vidyapeeth's College of Engineering, Lavale, Pune for their constant support and encouragement.

We would like to express our thanks to **Prof. Dr. R. N. Patil** Principal, Bharati Vidyapeeth's College of Engineering, Lavale, Pune for permitting us to take up the project work. We are thankful to our parents and friends for their kind help and support throughout the course.

Purva Pramod Raut (30)
Student Name

ABSTRACT

The Snake and Ladder game is a popular board game that can be implemented using Data Structures and Algorithms. This project focuses on the implementation of the Snake and Ladder game using the Breadth-First Search (BFS) algorithm in Python. The main objective of this project is to find the shortest path from the starting position to the destination square on the board.

In this project, the game board is represented as a graph in which each square represents a vertex and the possible dice moves represent edges. Snakes move the player to a lower-numbered square, while ladders move the player to a higher-numbered square. The BFS algorithm explores the possible moves level by level using a queue and keeps track of visited squares to avoid repeated visits.

The algorithm calculates the minimum number of dice throws required to reach the final square, assuming that each dice outcome from one to six is equally possible as a legal move and that every move has equal cost. The project demonstrates the practical application of graph traversal, queues, and shortest-path finding.

This project helps in understanding how BFS can be used to solve real-world problems through efficient searching techniques. It also improves knowledge of Python programming and fundamental data structures.

TABLE OF CONTENTS

SR. NO.	CONTENT	PAGE NO.
1	Introduction a) Overview c) Objective	1
2	Literature Survey	2
3	UML / Use Case Diagram	3
4	System Requirements	4
5	System Design a) Existing System b) Proposed System c) Activity Diagram d) System Code	5
6	Result and Testing	7
7	Applications	12
8	Future Scope	13
9	Conclusion	14
10	References	15

LIST OF FIGURES

Figure	Description	Page No.
Fig 4.1	UML Case Diagram	03
Fig 5.1	System Design	06
Fig 5.2	System Code	07
Fig 5.3	Result	8

LIST OF TABLES

Table No.	Table Name	Page No.
2.1	Literature Survey	2

INTRODUCTION

Overview:

Snake and Ladder is a popular board game in which players move from the starting square to the destination by rolling a dice. The board contains snakes that move players backward and ladders that help players move forward. This project implements the Snake and Ladder game using the Breadth-First Search (BFS) algorithm in Python.

BFS is a graph traversal algorithm that explores nodes level by level using a queue. In this project, each square on the board is represented as a node, and possible dice moves are represented as edges. Snakes and ladders are treated as special movements between squares. BFS explores all possible moves and finds the shortest path from the starting square to the final square.

This project demonstrates the practical use of graph theory, queues, and shortest-path algorithms in solving a familiar board game.

Objective:

The main objective of this project is to implement the Snake and Ladder problem using the Breadth-First Search (BFS) algorithm. The specific objectives are:

- To understand the concept of graphs and graph traversal.
- To implement BFS using a queue in Python.
- To represent the Snake and Ladder board as a graph.
- To handle snake and ladder movements during traversal.
- To find the minimum number of dice throws required to reach the destination.
- To understand the practical application of data structures and algorithms in problem-solving.

LITERATURE SURVEY

Year	Paper Name	Author	Conclusion
2001	Introduction to Algorithms	Cormen, Leiserson, Rivest and Stein	This book explains Breadth-First Search (BFS), graph traversal, and shortest-path finding. It provides the theoretical foundation for solving the Snake and Ladder problem.
2011	Algorithms	Robert Sedgewick and Kevin Wayne	This book describes graph algorithms and efficient ways to explore connected nodes. These concepts help represent the board as a graph and process possible moves.
2014	Introduction to Graph Theory	Douglas B. West	This book explains vertices, edges, and graph structures. These concepts are useful for representing board squares and the movements caused by snakes and ladders.
2016	Problem Solving with Algorithms and Data Structures Using Python	Brad Miller and David Ranum	This book discusses Python implementations of data structures, queues, and graph algorithms. It supports the practical implementation of BFS using Python.

UML DIAGRAM

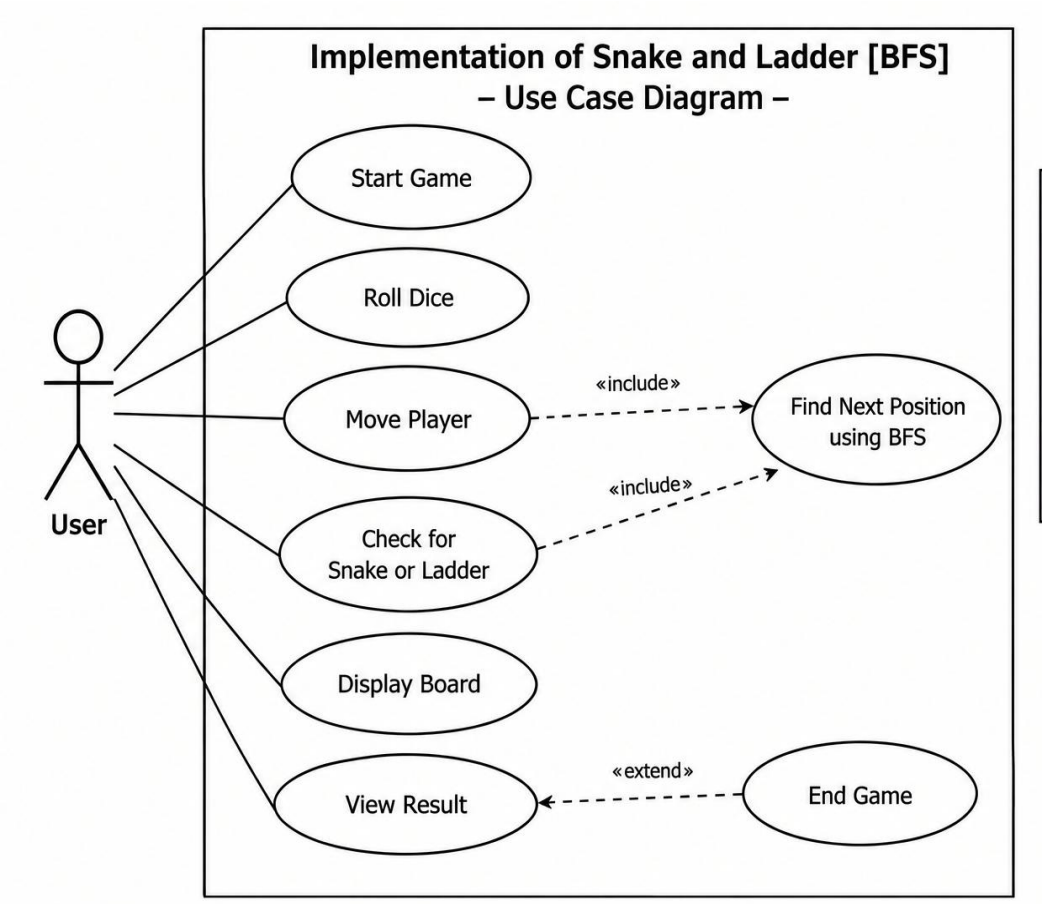


Fig 3.1: UML Diagram

SYSTEM REQUIREMENT

Hardware Requirements

- Processor: Intel Core i3 or equivalent
- RAM: Minimum 4 GB
- Hard Disk: 500 MB free space
- Input Device: Keyboard and mouse
- Output Device: Monitor

Software Requirements

- Operating System: Windows 10 / Windows 11
- Programming Language: Python
- Development Environment: Jupyter Notebook
- Required Library: collections (for queue operations)
- Python Interpreter: Python 3.x

SYSTEM DESIGN

Existing System

In a traditional browsing system, users visit multiple web pages and use the back button to return to previously visited pages. Managing browsing history without a suitable data structure can make navigation difficult.

The existing system has the following limitations:

1. **History Management:** Managing visited pages can be difficult.
2. **Navigation Issues:** Returning to previous pages requires an organized history.
3. **Incorrect Order:** Pages may not be retrieved in the correct order without a suitable mechanism.
4. **Limited Efficiency:** Managing browsing history becomes complex as the number of pages increases.

Proposed System

The proposed system is a Python-based, menu-driven application that uses the Stack data structure to implement browser back button functionality.

It follows the Last In, First Out (LIFO) principle. When a user visits a new page, the page is pushed onto the stack. When the back button is pressed, the current page is popped from the stack, allowing the user to return to the previous page.

The proposed system provides the following features:

1. **Page Navigation:** Allows users to visit new web pages.
2. **Stack Management:** Stores visited pages using the push operation.
3. **Back Button:** Uses the pop operation to return to the previous page.
4. **History Display:** Displays pages stored in the stack.

Entity Relationship Diagram:

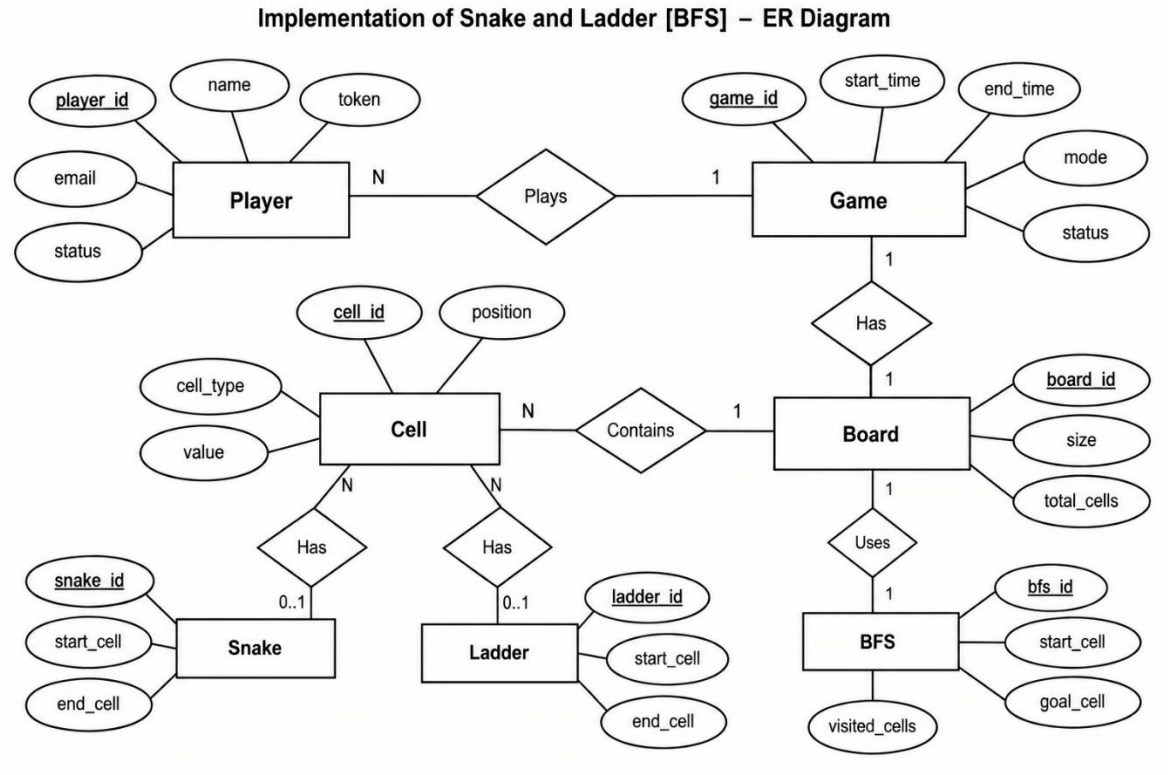
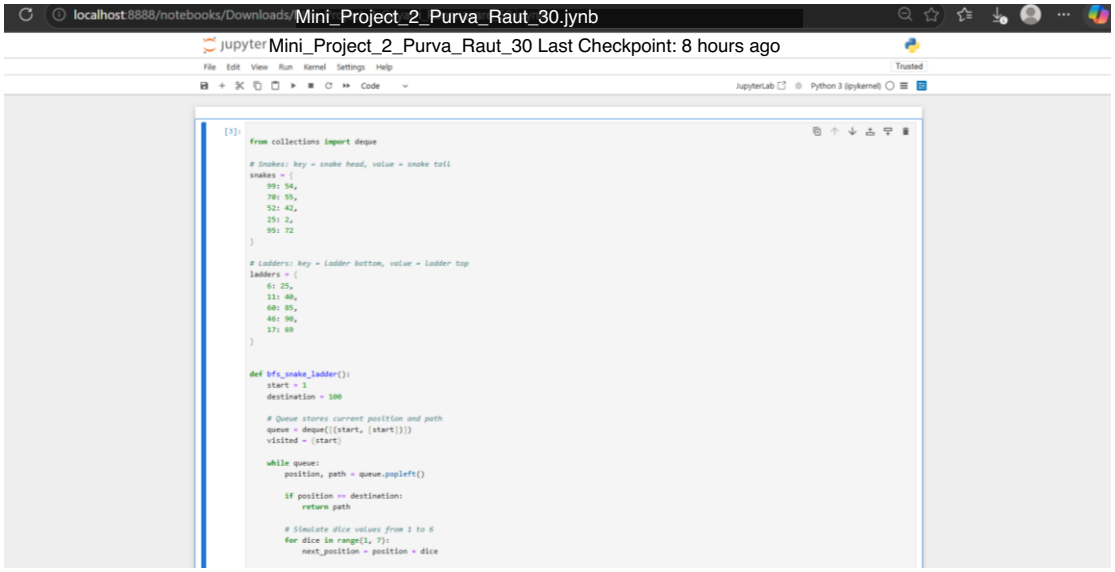
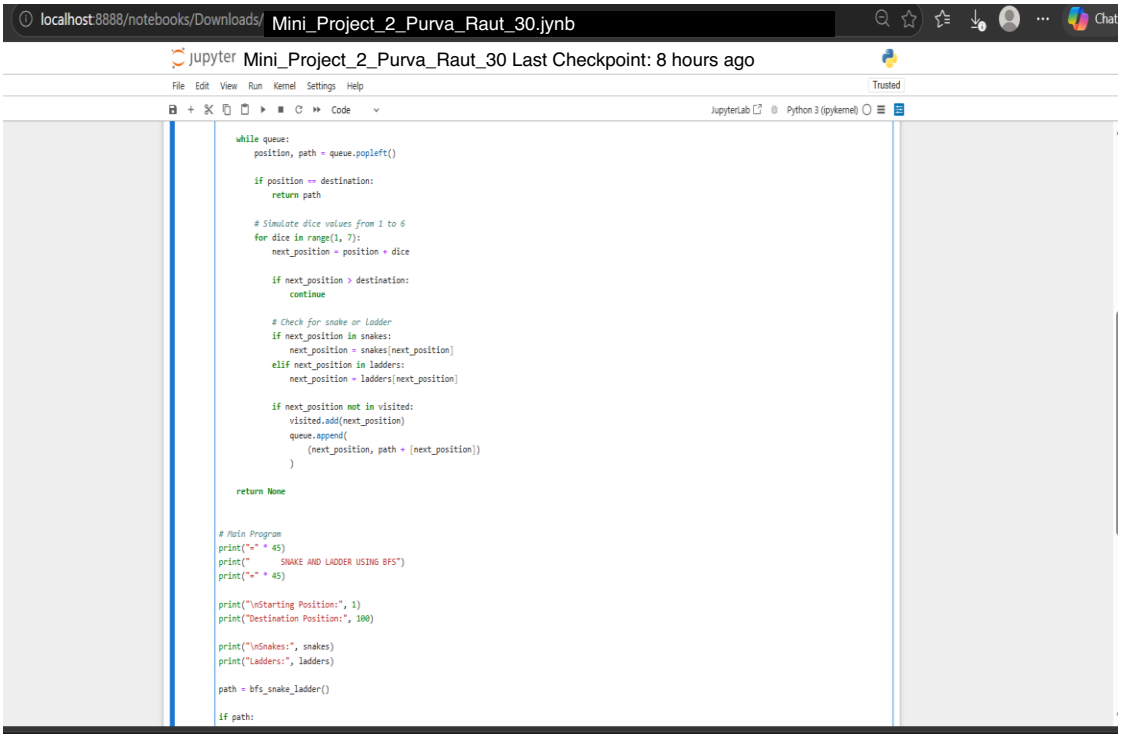


Fig 5.1: ER Diagram

SYSTEM CODE


localhost:8888/notebooks/Downloads/Mini_Project_2_Purva_Raut_30.jynb
jupyter Mini_Project_2_Purva_Raut_30 Last Checkpoint: 8 hours ago
File Edit View Run Kernel Settings Help
Trusted
Code
JupyterLab
Python 3 (ipykernel)
[3]:
from collections import deque
Snakes: key = snake head, value = snake tail
snakes = {
99: 54,
70: 55,
52: 42,
25: 2,
95: 72
}
Ladders: key = ladder bottom, value = ladder top
ladders = {
6: 25,
11: 40,
60: 85,
46: 90,
17: 69
}
def bfs_snake_ladder():
start = 1
destination = 100
Queue stores current position and path
queue = deque([(start, [start])])
visited = {start}
while queue:
position, path = queue.popleft()
if position == destination:
return path
Simulate dice values from 1 to 6
for dice in range(1, 7):
next_position = position + dice


localhost:8888/notebooks/Downloads/Mini_Project_2_Purva_Raut_30.jynb
jupyter Mini_Project_2_Purva_Raut_30 Last Checkpoint: 8 hours ago
File Edit View Run Kernel Settings Help
Trusted
Code
JupyterLab
Python 3 (ipykernel)
while queue:
position, path = queue.popleft()
if position == destination:
return path
Simulate dice values from 1 to 6
for dice in range(1, 7):
next_position = position + dice
if next_position > destination:
continue
Check for snake or ladder
if next_position in snakes:
next_position = snakes[next_position]
elif next_position in ladders:
next_position = ladders[next_position]
if next_position not in visited:
visited.add(next_position)
queue.append(
(next_position, path + [next_position])
)
return None
Main Program
print("=" * 45)
print(" SNAKE AND LADDER USING BFS")
print("=" * 45)
print("\nStarting Position:", 1)
print("Destination Position:", 100)
print("\nSnakes:", snakes)
print("Ladders:", ladders)
path = bfs_snake_ladder()
if path:

```
# Main Program
print("=" * 45)
print("       SNAKE AND LADDER USING BFS")
print("=" * 45)

print("\nStarting Position:", 1)
print("Destination Position:", 100)

print("\nSnakes:", snakes)
print("Ladders:", ladders)

path = bfs_snake_ladder()

if path:
    print("\nShortest Path:")
    print(" -> ".join(map(str, path)))

    print("\nTotal Moves Required:", len(path) - 1)
else:
    print("\nDestination cannot be reached.")
```

```
=============================================
```

Fig 5.2: System code

RESULT

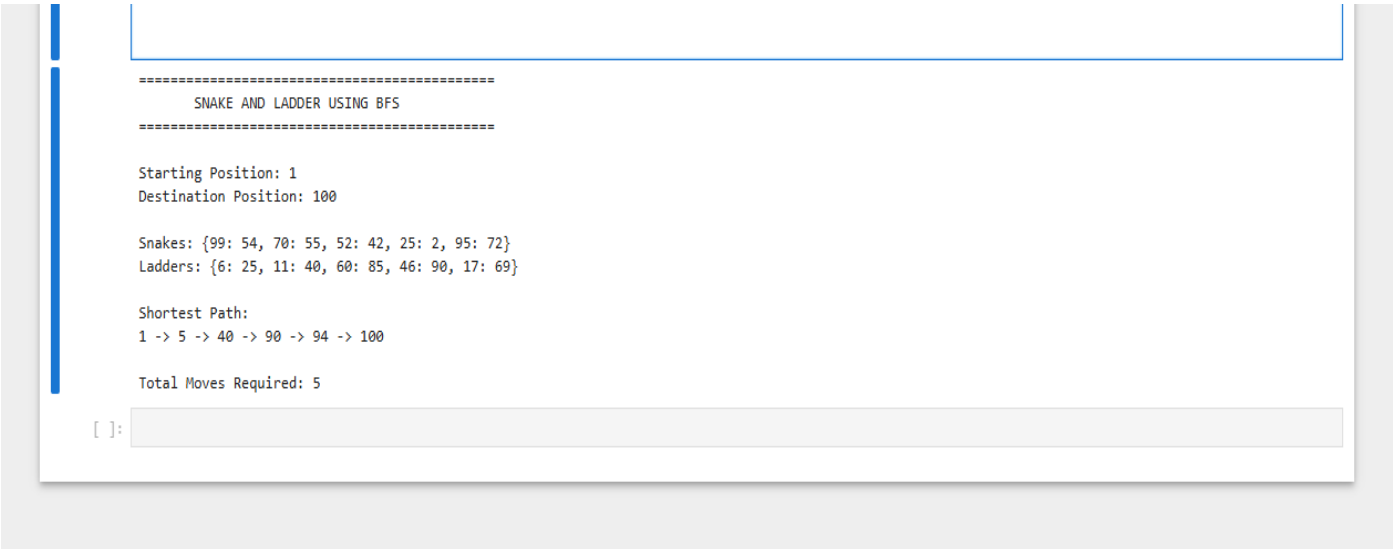


Fig 5.3 Result

APPLICATIONS

The Snake and Ladder game implemented using the Breadth-First Search (BFS) algorithm has the following applications:

1. Shortest Path Finding: BFS helps find the minimum number of dice throws required to reach the destination.
2. Learning Data Structures: The project helps students understand graphs, queues, sets, and dictionaries.
3. Game Development: Graph traversal techniques can be used to analyze possible moves in board games.
4. Artificial Intelligence: Search algorithms help computers explore possible states and select suitable paths in games.
5. Problem Solving: The project demonstrates how complex problems can be represented as graphs and solved systematically.
6. Algorithm Analysis: It helps students understand the time and space complexity of BFS.
7. Educational Purposes: This project provides a practical demonstration of graph traversal and shortest-path algorithms using Python.

FUTURE WORK

The Snake and Ladder game implemented using the Breadth-First Search (BFS) algorithm can be improved in several ways in the future.

1. Graphical User Interface (GUI): A graphical interface can be developed to make the game more attractive and user-friendly.
2. Multiplayer Mode: The game can be extended to support multiple players playing against each other.
3. Dice Animation: Animated dice rolling can be added to improve the gaming experience.
4. Interactive Board: A visual board can be developed to show player movements, snakes, and ladders.
5. Performance Improvement: Different search algorithms, such as Depth-First Search (DFS) and Dijkstra's algorithm, can be compared with BFS for educational purposes.
6. Score Tracking: The system can record the number of dice throws and display game statistics.
7. Mobile Application: The project can be converted into a mobile application for Android devices.

These improvements can make the project more interactive and useful while providing further opportunities to learn about data structures, algorithms, and game development.

.

CONCLUSION

The project “Implementation of Snake and Ladder Using BFS” demonstrates the practical application of Data Structures and Algorithms using Python. The game board is represented as a graph, where each square represents a node and possible dice moves represent edges. Snakes and ladders are handled as special transitions between squares.

The Breadth-First Search (BFS) algorithm explores possible moves level by level using a queue and finds the shortest path from the starting square to the destination. The program calculates the minimum number of dice throws required to reach square 100 and displays the path followed.

This project helps in understanding important concepts such as graph traversal, queue operations, visited nodes, and shortest-path finding. It also improves programming and problem-solving skills through practical implementation.

In conclusion, the project successfully demonstrates how BFS can be used to solve the Snake and Ladder problem efficiently and provides a strong foundation for understanding graph-based algorithms and their applications in computer science.

REFERENCE

1. Cormen, T. H., Leiserson, C. E., Rivest, R. L., and Stein, C. *Introduction to Algorithms*. 3rd Edition, MIT Press, 2009.

2. Sedgewick, R., and Wayne, K. *Algorithms*. 4th Edition, Addison-Wesley Professional, 2011.

3. Bhargava, A. *Grokking Algorithms: An Illustrated Guide for Programmers and Other Curious People*. Manning Publications, 2016.

4. Miller, B., and Ranum, D. *Problem Solving with Algorithms and Data Structures Using Python*. Runestone Academy.

5. Python Software Foundation. *Python 3 Documentation*. Available at: https://docs.python.org/3/

6. Jupyter Project. *Jupyter Notebook Documentation*. Available at: https://jupyter-notebook.readthedocs.io/

7. GeeksforGeeks. *Breadth First Search or BFS for a Graph*. Available at: https://www.geeksforgeeks.org/dsa/breadth-first-search-or-bfs-for-a-graph/